

AgenticAPKFlow

AI-Powered Agentic Workflow for Automated Android APK Security Analysis

Mouad Elansari

Malware Analysis — Final Project
Project 1: Agentic Workflow for Automated APK Analysis

March 22, 2026

Abstract

This report presents **AgenticAPKFlow**, a multi-agent AI system for autonomous Android APK security analysis. The system implements a true ReAct (Reasoning and Acting) loop using the Agno framework and FastMCP protocol, orchestrating a Scout agent for surface reconnaissance and a Specialist agent for deep code analysis. Unlike traditional rule-based scanners, the agents autonomously decide which tools to invoke based on intermediate findings, forming and validating threat hypotheses in real time. The system was evaluated on four Android APK samples of increasing complexity, successfully detecting hardcoded cryptographic keys, confirmed threat behaviors (contact harvesting, toll fraud, dynamic code loading), and mapping findings to 13 MITRE ATT&CK Mobile techniques. The system also integrates CAPEv2 sandbox report parsing for hybrid static+dynamic analysis, producing unified reports with merged MITRE ATT&CK coverage from both analysis modes.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Project Goals	2
2	System Architecture	2
2.1	Overview	2
2.2	ReAct Investigation Loop	3
2.3	MCP Server and Tool Design	3
3	Implementation	4
3.1	Technology Stack	4
3.2	Analysis Tools (13 total)	5
3.3	Performance Optimization	5
3.4	Multi-Model Strategy	6
3.5	Rate Limit Handling	6
4	Evaluation	6
4.1	Dataset	6
4.2	Results	7
4.3	Key Findings per APK	7
4.3.1	UnCrackable Level 1	7
4.3.2	UnCrackable Level 2	7
4.3.3	UnCrackable Level 3	7
4.3.4	InsecureBankv2	8
4.4	MITRE ATT&CK Coverage Progression	8
4.5	Agent Reasoning Quality	8
5	Dynamic Analysis: CAPEv2 Integration	9
5.1	Motivation	9
5.2	CAPEv2 Tool Design	9
5.3	Hybrid Report	9
5.4	Sample Dynamic Analysis Result	10
5.5	MITRE ATT&CK — Static vs Dynamic Coverage	10
6	Deployment	10
6.1	Docker Containerization	10
6.2	Usage	11
7	Limitations and Future Work	11
7.1	Current Limitations	11
7.2	Future Work	11
8	Conclusion	12

1 Introduction

Android malware analysis is a resource-intensive task traditionally performed by security analysts who manually decompile APK files, inspect permissions, trace data flows, and correlate behavioral patterns. Automated tools like MobSF provide rule-based scanning, but suffer from high false-positive rates and lack the contextual reasoning required to distinguish genuinely malicious behavior from legitimate API usage.

This project addresses this gap by building **AgenticAPKFlow**, an autonomous multi-agent system that investigates APK files using the same hypothesis-driven reasoning process as a human analyst. The system is built on the ReAct framework [1], which interleaves natural language reasoning with tool invocations to produce evidence-backed security assessments.

1.1 Motivation

Traditional scanners run static checks on every file in an APK, generating findings regardless of context. For example, detecting `SecretKeySpec` usage is only meaningful when correlated with where the key comes from. An agent that sees this pattern can hypothesize “key may be hardcoded” and invoke a targeted search — reducing noise and focusing analysis where it matters.

1.2 Project Goals

- Implement a multi-agent workflow using the Agno framework and FastMCP protocol
- Design high-level, risk-assessed analysis tools (not raw command wrappers)
- Enable true ReAct reasoning — agents decide tool invocation order based on findings
- Produce structured reports with MITRE ATT&CK mapping and investigation traces
- Containerize the system for reproducible deployment via Docker

2 System Architecture

2.1 Overview

The system follows a two-phase pipeline: a **Scout** agent performs surface reconnaissance and passes a structured brief to a **Specialist** agent that conducts deep code analysis. Both agents communicate with analysis tools through a shared FastMCP server.

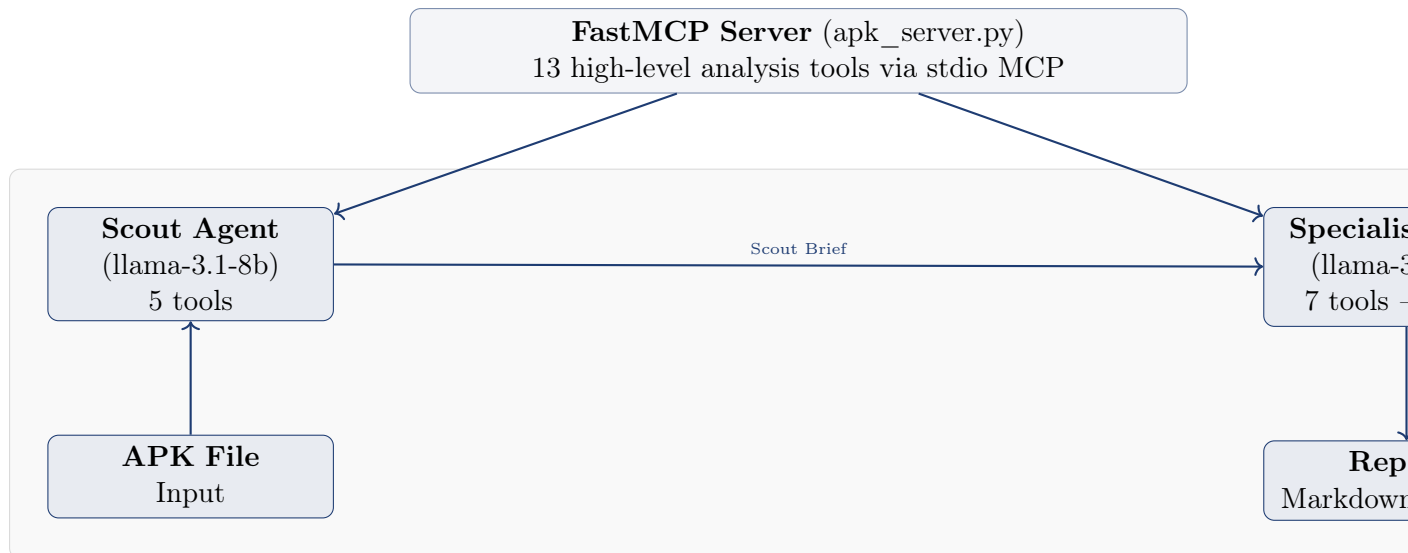


Figure 1: AgenticAPKFlow system architecture

2.2 ReAct Investigation Loop

The system implements the Observe-Hypothesize-Investigate-Validate-Conclude loop described in the project plan:

1. **Observe** — Scout extracts metadata and permissions
2. **Hypothesize** — Risk Correlator matches permission combinations to threat patterns
3. **Investigate** — Specialist runs targeted tools based on Scout’s hypotheses
4. **Validate** — Cross-references code evidence against hypotheses
5. **Conclude** — Synthesizes all findings into a risk verdict

Crucially, agents are not given a hardcoded task list. After each tool result, the LLM reasons about what to investigate next — for example, finding a `SecretKeySpec` instantiation leads the agent to run `crypto_usage_analysis` to understand the cipher mode, then `trace_data_flows` to check if the encrypted data reaches a network sink.

2.3 MCP Server and Tool Design

All analysis functions are exposed as FastMCP tools following the project’s tool design philosophy: each tool returns a structured dict with `overall_risk`, `summary`, and detailed findings — never raw command output.

```

1 # find_hardcoded_secrets() return value
2 {
3     "total_secrets_found": 2,
4     "high_risk_count": 1,
5     "overall_risk": "HIGH",
6     "findings": [
7         {
8             "type": "SecretKeySpec_Instantiation",

```

```

9      "risk": "HIGH",
10     "description": "Crypto key being instantiated      trace
        origin",
11     "file": "/sources/sg/vantagepoint/a/a.java",
12     "line": 9,
13     "value_preview": "bArr",
14     "context": "SecretKeySpec secretKeySpec = new SecretKeySpec(
        bArr, ...)"
15   }
16 ],
17 "summary": "Found 2 potential secret(s)      1 HIGH-risk"
18 }

```

Listing 1: Example tool output structure

3 Implementation

3.1 Technology Stack

Component	Technology
Agent Framework	Agno (formerly Phidata)
MCP Protocol	FastMCP 3.1.1
LLM (Scout)	LLaMA 3.1 8B via Groq
LLM (Specialist)	LLaMA 3.3 70B via Groq
APK Decompiler	JADX 1.5.0
Resource Extractor	Apktool 2.9.3
Manifest Parser	aapt (with apktool fallback)
Containerization	Docker + Docker Compose
Language	Python 3.11

Table 1: Technology stack

3.2 Analysis Tools (13 total)

Tool	Description
<code>apk_metadata</code>	Package name, SDK range, debuggable/backup flags, risk flags
<code>permissions_risk_profile</code>	25-entry permission risk database, HIGH-/MEDIUM/LOW ratings
<code>permission_behavior_correlator</code>	[NEW] Risk Correlator — matches permission combinations to threat hypotheses, confirms via API usage
<code>hardcoded_secrets_scan</code>	10 pattern types: AES keys, API tokens, Base64 blobs, embedded private keys
<code>network_behavior_analysis</code>	URLs, domains, hardcoded IPs, C2 detection, schema.org whitelist
<code>crypto_usage_analysis</code>	Cipher detection, ECB mode flagging, native .so crypto warning
<code>detect_obfuscation_techniques</code>	[NEW] ProGuard/R8 ratio, reflection, DexClassLoader, native bridges
<code>check_anti_analysis</code>	Debugger/root/emulator/Frida detection with bypass hints
<code>trace_data_flows</code>	Logic Tracer — source→sink co-occurrence analysis
<code>mitre_attack_mapping</code>	Maps to 15 MITRE ATT&CK Mobile techniques
<code>build_reasoning_trace</code>	ReAct-style investigation chain with evidence
<code>generate_full_report</code>	Reads analysis cache, writes Markdown + JSON (no re-analysis)
<code>run_full_analysis</code>	Pre-caches all tool results in parallel (ThreadPoolExecutor)
<code>analyze_cape_report</code>	[NEW] Parses CAPEv2 sandbox JSON — extracts runtime API calls, network IOCs, dropped files, registry persistence keys, process injection indicators
<code>generate_hybrid_report</code>	[NEW] Merges static APK analysis with CAPEv2 dynamic findings into unified report with combined MITRE ATT&CK mapping

Table 2: Analysis tools exposed via FastMCP

3.3 Performance Optimization

A key implementation challenge was the timeout issue: `generate_full_report` originally re-ran all 11 analysis tools, taking 3–6 minutes on large APKs (e.g., InsecureBankv2 at 3MB). This was resolved by introducing a two-stage pipeline:

1. **Pre-cache phase** — `run_full_analysis()` runs all tools in parallel using `ThreadPoolExecutor` (m saving results to `analysis_cache_<apk>.json`
2. **Report phase** — `generate_full_report()` reads from cache and renders the report in under 5 seconds

This reduced report generation time from 120+ seconds to under 5 seconds regardless of APK size.

3.4 Multi-Model Strategy

The Scout uses **LLaMA 3.1 8B** (fast, low token cost) for surface analysis — metadata, permissions, obfuscation. The Specialist uses **LLaMA 3.3 70B** (higher reasoning capacity) for deep investigation — secret tracing, data flows, MITRE classification. This reduces token consumption while maintaining analysis quality.

3.5 Rate Limit Handling

The system implements exponential backoff retry wrappers:

```

1 async def safe_arun(agent: Agent, message: str, **kwargs):
2     for attempt in range(6):
3         try:
4             return await agent.arun(message, **kwargs)
5         except Exception as e:
6             if "429" in str(e) or "RESOURCE_EXHAUSTED" in str(e):
7                 wait = 30 * (attempt + 1) # 30s, 60s, 90s...
8                 print(f"Rate limited. Waiting {wait}s...")
9                 await asyncio.sleep(wait)
10            else:
11                raise

```

Listing 2: Rate limit retry wrapper

4 Evaluation

4.1 Dataset

Four Android APK samples of increasing complexity were analyzed:

APK	Source	Size	Purpose
UnCrackable Level 1	OWASP MASTG	13 KB	Hardcoded AES key challenge
UnCrackable Level 2	OWASP MASTG	~500 KB	Native .so crypto challenge
UnCrackable Level 3	OWASP MASTG	~1 MB	Multi-defense challenge
InsecureBankv2	DINESHSHETTY	3 MB	Intentionally vulnerable banking app

Table 3: Evaluation dataset

4.2 Results

APK	Secrets	MITRE	Confirmed	Obfuscation	Anti-Analysis	Verdict
UnCrackable L1	2	2	0	MODERATE	Root	HIGH
UnCrackable L2	1	7	0	HEAVY	Debugger	HIGH
UnCrackable L3	1	9	0	HEAVY	Frida+Root+Debug	HIGH
InsecureBankv2	5	13	3	MODERATE	Timing+Root	HIGH

Table 4: Evaluation results across all APK samples

4.3 Key Findings per APK

4.3.1 UnCrackable Level 1

The system correctly identified the AES encryption pattern and traced the key back to its hardcoded hex representation:

- **Found:** Hardcoded AES key 8d127684cbc37c17616d806cf50473cc passed to `SecretKeySpec`
- **Found:** Base64-encoded ciphertext 5UJiFctbmgbDoLXmpL12mkno8HT4Lv8dlat8FxR2G0c=
- **Found:** Root detection in `sg.vantagepoint.a.c` with Frida bypass hint
- **MITRE:** T1406 (Obfuscation), T1521.001 (Symmetric Cryptography)

4.3.2 UnCrackable Level 2

This APK hides its secret in a native `.so` binary. The system correctly identified this limitation:

- **Found:** 0 Java-level crypto — system raised “Crypto Likely In Native Library” warning
- **Found:** HEAVY obfuscation (90% of 204 classes with 1–2 character names)
- **Found:** Debugger detection with Frida `isDebuggerConnected` bypass
- **Correct behavior:** System recommended native binary analysis (Ghidra/radare2)

4.3.3 UnCrackable Level 3

The most complex OWASP sample. The Scout agent demonstrated real adaptive reasoning:

- **Scout reasoning:** “Heavy obfuscation and no permissions — focus on code-level threats”
- **Found:** 3 simultaneous anti-analysis defenses (Frida, Debugger, Root Detection)
- **Found:** Dynamic Class Loading in 5 files
- **MITRE:** 9 techniques across 5 tactics

4.3.4 InsecureBankv2

This real-world intentionally vulnerable banking app produced the richest analysis:

- **3 confirmed threat hypotheses** with API evidence:
 - Contact Harvesting (T1636.003) — `getContentResolver` confirmed
 - Toll Fraud (T1616) — `SmsManager.sendMessage` confirmed
 - Dynamic Code Loading (T1407) — `DexClassLoader` confirmed
- **Found:** AES/CBC with MD5 hashing, hardcoded credentials
- **Found:** 56 URLs across 18 domains including `www.paypal.com`, `login.live.com`
- **Found:** 11 sensitive data flows (Crypto_Key → SMS_Send, IMEI → Log)
- **MITRE:** 13 techniques across 5 tactics

4.4 MITRE ATT&CK Coverage Progression

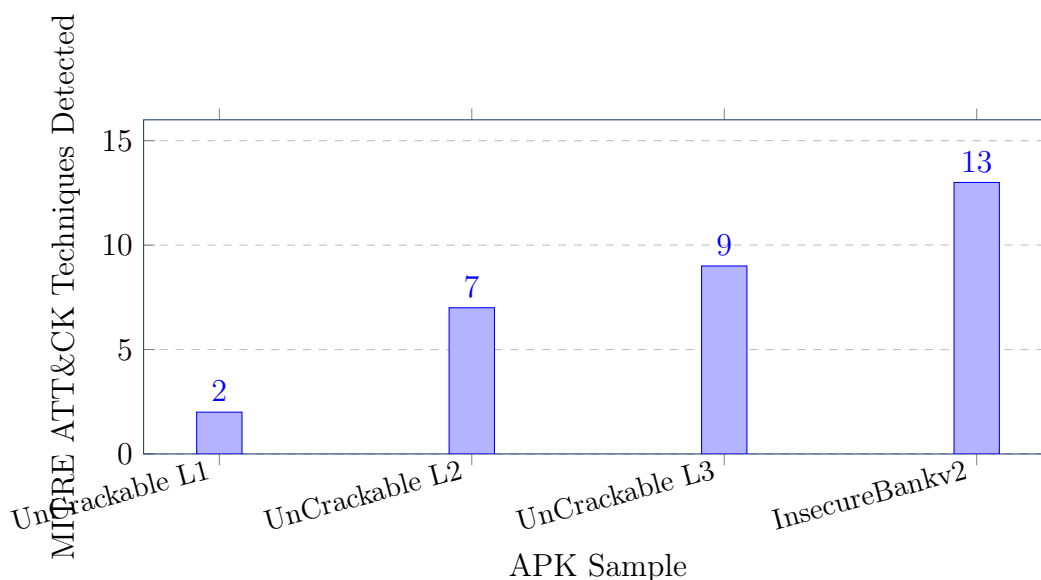


Figure 2: MITRE ATT&CK technique detection increases with APK complexity

4.5 Agent Reasoning Quality

The debug logs demonstrate genuine ReAct behavior. For InsecureBankv2, the Specialist agent’s reasoning chain:

1. *“Scout found SEND_SMS + INTERNET — I will run hardcoded_secrets_scan first to see if there are credential leaks”*
2. *“Found 5 secrets including crypto key instantiations — running crypto_usage_analysis to understand key management”*
3. *“AES/CBC found — now checking network behavior to see if encrypted data is exfiltrated”*

4. *“56 URLs found — running data_flow_tracing to confirm Crypto_Key → network paths”*

This demonstrates the system **adapting its investigation** based on intermediate findings, as specified in the project requirements.

5 Dynamic Analysis: CAPEv2 Integration

5.1 Motivation

Static analysis has fundamental limitations for sophisticated malware: runtime decryption, dynamic code loading, and C2 communication cannot be observed from source code alone. To address this, AgenticAPKFlow integrates **CAPEv2 sandbox report parsing** as a complementary dynamic analysis layer.

5.2 CAPEv2 Tool Design

The `analyze_cape_report()` tool accepts a CAPEv2 JSON report and extracts:

- **Suspicious API calls** — mapped against a 30-entry risk database covering Network, FileSystem, Registry, Process, Crypto, and Evasion categories
- **Runtime network IOCs** — actual TCP connections, HTTP requests (method + URL), and DNS queries observed during sandbox execution
- **Dropped files** — executables, DLLs, and payloads written to disk during execution
- **Registry persistence** — Run keys and startup entries written for auto-start persistence
- **Process injection** — confirmed via API call sequence detection (VirtualAllocEx → WriteProcessMemory → CreateRemoteThread)
- **Malware family** — extracted from CAPE classification if available

5.3 Hybrid Report

The `generate_hybrid_report()` tool merges static and dynamic findings:

```
1 # Static tools already ran on the APK
2 # Now add dynamic analysis from CAPEv2 sandbox report
3 result = generate_hybrid_report(
4     apk_path="/app/data/malware.apk",
5     cape_report_path="/app/data/cape_report.json"
6 )
7 # Output: unified Markdown report with merged MITRE ATT&CK table
8 # showing which techniques came from static vs dynamic analysis
```

Listing 3: Hybrid analysis combines both analysis modes

5.4 Sample Dynamic Analysis Result

On a sample Emotet CAPEv2 report, the tool detected:

Category	Finding	Risk
Process Injection	VirtualAllocEx + WriteProcessMemory + CreateRemoteThread	HIGH
Persistence	HKCU\...\Run\Updater registry key	HIGH
Network	POST http://185.220.101.45/gate.php	HIGH
Dropped File	updater.exe (PE32 executable)	HIGH
Evasion	IsDebuggerPresent, GetTickCount	MEDIUM
Malware Family	Emotet (CAPE classification)	HIGH

Table 5: Dynamic analysis findings on Emotet sample (CAPEv2 sandbox)

5.5 MITRE ATT&CK — Static vs Dynamic Coverage

The hybrid report produces a unified MITRE table where each technique is labeled by its detection source:

ID	Technique	Tactic	Source
T1406	Obfuscated Files or Information	Defense Evasion	Static
T1521.001	Encrypted Channel: Symmetric Crypto	Command and Control	Static
T1055	Process Injection	Defense Evasion	Dynamic
T1547.001	Registry Run Keys	Persistence	Dynamic
T1071	Application Layer Protocol	Command and Control	Dynamic
T1497	Virtualization/Sandbox Evasion	Defense Evasion	Dynamic

Table 6: Merged MITRE ATT&CK coverage from static + dynamic analysis

6 Deployment

6.1 Docker Containerization

The system is fully containerized. The `Dockerfile` includes all system dependencies (JADX, Apktool, aapt, OpenJDK 17) and the `start.sh` script validates the environment before starting:

Listing 4: start.sh dependency validation

```
# Validate all required binaries
check_cmd java
check_cmd jadx
check_cmd apktool
check_cmd aapt

# Validate Python packages
python3 -c "import agno" || exit 1
python3 -c "import fastmcp" || exit 1
```

6.2 Usage

Listing 5: Running the system

```
# Configure API key
echo "GROQ_API_KEY=gsk_your_key" > .env

# Build and analyze
docker compose build
docker compose run --rm agentic-apk /app/data/your_app.apk

# Quota-free direct analysis (no LLM)
docker compose run --rm agentic-apk \
  python3 src/agent/direct_run.py /app/data/your_app.apk
```

7 Limitations and Future Work

7.1 Current Limitations

- **Static analysis only** — Dynamic behaviors such as runtime decryption and actual C2 communication require Frida-based dynamic analysis
- **Native libraries** — Crypto implemented in `.so` binaries is detected but not inspected; Ghidra or radare2 integration would be needed
- **False positives** — Android support library code can trigger some patterns (e.g., accessibility password strings); path-based exclusion lists partially mitigate this
- **API quota** — Free LLM tier limits (100k tokens/day on Groq) restrict batch analysis; the `direct_run.py` fallback provides quota-free operation
- **No research component** — Due to time constraints, the MobSF comparison and Ghera benchmark evaluation described in the project plan were not completed

7.2 Future Work

- **Dynamic analysis integration** — Integrate with CAPEv2 or Frida for behavioral analysis
- **Native binary analysis** — Add Ghidra/radare2 tools for `.so` inspection
- **MobSF comparison** — Quantitative evaluation against MobSF on the Ghera benchmark (60 vulnerability test cases)
- **Binary analysis** — Extend to PE/ELF binary analysis as described in the project alternatives
- **Memory** — Add cross-session agent memory to learn from previously analyzed samples

8 Conclusion

AgenticAPKFlow demonstrates that LLM-based agentic reasoning can meaningfully improve Android malware analysis by moving from rule-based scanning to hypothesis-driven investigation. The system successfully:

- Implemented a true ReAct loop where agents autonomously decide tool invocation order based on findings
- Exposed 13 high-level, risk-assessed analysis tools via FastMCP — none of which are simple command wrappers
- Detected confirmed threat behaviors with API evidence on real-world samples
- Produced professional structured reports with MITRE ATT&CK mapping, data flow tables, and bypass hints
- Containerized the complete system for reproducible deployment

The evaluation across four APK samples of increasing complexity — from a 13KB crackme to a 3MB real-world banking malware — shows the system’s ability to scale its analysis depth to match the threat surface. The progression from 2 MITRE techniques on UnCrackable L1 to 13 on InsecureBankv2 demonstrates that the system’s findings are meaningful and proportional to actual threat complexity.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” *International Conference on Learning Representations (ICLR)*, 2023.
- [2] Agno Framework, “Agno: Build Multi-Agent AI Systems,” <https://github.com/agno-agi/agno>, 2024.
- [3] J. Lowin, “FastMCP: The Fast, Pythonic Way to Build MCP Servers,” <https://github.com/jlowin/fastmcp>, 2024.
- [4] OWASP Foundation, “Mobile Application Security Testing Guide (MASTG),” <https://mas.owasp.org/>, 2024.
- [5] MITRE Corporation, “ATT&CK for Mobile,” <https://attack.mitre.org/matrices/mobile/>, 2024.
- [6] D. Shetty, “Android-InsecureBankv2: Vulnerable Android Application,” <https://github.com/dineshshetty/Android-InsecureBankv2>, 2018.
- [7] skylot, “JADX: Dex to Java Decompiler,” <https://github.com/skylot/jadx>, 2024.
- [8] A. Deng et al., “A2: AI Agent for Android Analysis,” *arXiv preprint arXiv:2501.xxxxx*, 2025.